



**TRIBHUVAN UNIVERSITY
INSTITUTE OF ENGINEERING
PURWANCHAL CAMPUS**

**A MINOR PROJECT REPORT ON
INTELLIGENT PARKING BILLING SYSTEM USING YOLO AND OCR**

BY

**PRANIKA ANGDEMBE LIMBU (PUR079BCT051)
RAGHAV UPADHYAY (PUR079BCT057)
SANDIP SAPKOTA (PUR079BCT076)
SULAV POUDEL (PUR079BCT087)**

**DEPARTMENT OF ELECTRONICS AND COMPUTER ENGINEERING
PURWANCHAL CAMPUS
DHARAN, NEPAL**

APRIL, 2026

PAGE OF APPROVAL

TRIBHUVAN UNIVERSITY
INSTITUTE OF ENGINEERING
PURWANCHAL CAMPUS
DEPARTMENT OF ELECTRONICS AND COMPUTER ENGINEERING

The undersigned certifies that they have read and recommended to the Institute of Engineering for acceptance of a project report entitled “**INTELLIGENT PARKING BILLING SYSTEM USING YOLO AND OCR**” submitted by **PRANIKA ANGDEMBE LIMBU, RAGHAV UPADHYAY, SANDIP SAPKOTA and SULAV POUDEL** in partial fulfillment of the requirements for the Bachelor’s degree in Electronics & Computer Engineering.

.....

Supervisor

Er. Om Prakash Dhakal

Assistant Professor

Department of Electronics and Computer Engineering,

Purwanchal Campus, IOE, TU.

Date of approval:

ACKNOWLEDGEMENT

We express our sincere gratitude to everyone who supported and guided us during the completion of this project entitled “**INTELLIGENT PARKING BILLING SYSTEM USING YOLO AND OCR**”

First, we thank our respected supervisor **Er .OM PRAKASH DHAKAL** sir for his continuous guidance, valuable suggestions, and encouragement throughout the development of this project. His support and constructive feedback greatly helped us in successfully completing this work.

We would also like to express our heartfelt appreciation to the Head of the Department, **Er. MANOJ KUMAR GURAGAIN** of the Department of Electronics and Computer Engineering, Institute of Engineering, Tribhuvan University, for their support and motivation during our project work.

Finally, we would like to thank Purwanchal Campus, Dharan and the Institute of Engineering, Tribhuvan University, for providing a supportive academic environment, necessary resources, and facilities that made this project possible.

We are grateful to all those who directly or indirectly contributed to the successful completion of this project.

Pranika Angdembe Limbu [PUR079BCT051]

Raghav Upadhyay [PUR079BCT057]

Sandip Sapkota [PUR079BCT076]

Sulav Poudel [PUR079BCT087]

© COPYRIGHT

The author has agreed that the Library, Department of Electronics and Computer Engineering, Purwanchal Campus, Institute of Engineering may make this report freely available for inspection. Moreover, the author has agreed that permission for extensive copying of this project report for scholarly purpose may be granted by the supervisor(s) who supervised this work recorded herein or, in their absence, by the Head of the Department wherein the thesis report was done. It is understood that the recognition will be given to the author of this report and to the Department of Electronics and Computer Engineering, Purwanchal Campus, Institute of Engineering in any use of the material of this thesis report. Copying or publication or the other use of this report for financial gain without approval of the Department of Electronics and Computer Engineering, Purwanchal Campus, Institute of Engineering and author's written permission is prohibited.

Request for permission to copy or to make any other use of the material in this report in whole or in part should be addressed to:

Head

Department of Electronics and Computer Engineering

Purwanchal Campus, Institute of Engineering

Dharan , Sunsari

Nepal

DECLARATION

We declare that the work hereby submitted for Bachelors of Engineering in Computer Engineering at Institute of Engineering, Purwanchal Campus entitled **INTELLIGENT PARKING BILLING SYSTEM USING YOLO AND OCR** is our own work and has not been previously submitted by us at any university for any academic award.

We authorize Institute of Engineering, Purwanchal Campus to lend this report to other institution or individuals for the purpose of scholarly research.

Pranika Angdembe Limbu [PUR079BCT051]

Raghav Upadhyay [PUR079BCT057]

Sandip Sapkota [PUR079BCT076]

Sulav Poudel [PUR079BCT087]

April 2026

ABSTRACT

Automatic Number Plate Recognition (ANPR) for Nepali vehicle registration plates presents a distinct challenge owing to the use of the Devanagari script, which renders conventional OCR techniques ineffective. This project presents an end-to-end ANPR pipeline designed specifically for Nepali number plates. The system employs a two-stage YOLO-based detection architecture: the first model localises the number plate within a scene, and the second segments individual Devanagari characters from the detected plate region. Character crops are classified by a custom ResNet-18 model trained on a dataset of Nepali Devanagari character images, balanced via oversampling to address class imbalance. Preprocessing follows a standardised pipeline CLAHE contrast enhancement, Gaussian blur, and adaptive thresholding applied identically during training and inference to ensure domain alignment. Post-processing applies digit correction and format validation to produce a structured plate string. The proposed approach demonstrates reliable recognition of both single-row and dual-row Nepali plate formats

Keywords: *ANPR, Nepali number plate recognition, Devanagari OCR, YOLO, ResNet-18, real-time video processing.*

CONTENTS

PAGE OF APPROVAL	ii
ACKNOWLEDGEMENT	iii
© COPYRIGHT	iv
DECLARATION	v
ABSTRACT	vi
LIST OF FIGURES	ix
LIST OF TABLES	x
LIST OF ABBREVIATIONS	xi
1 INTRODUCTION	1
1.1 Background	1
1.2 Gap Identification	1
1.3 Motivation	2
1.4 Objectives	2
2 LITERATURE REVIEW	3
2.1 Related Work	3
2.2 Related Theory	5
2.2.1 Automatic Number Plate Recognition (ANPR)	5
2.2.2 Computer Vision and Object Detection	5
2.2.3 Convolutional Neural Networks (CNN)	6
2.2.4 Optical Character Recognition (OCR)	6
2.2.5 Database and Billing System	7
3 METHODOLOGY	8
3.1 Detection and Recognition Pipeline	15
3.2 Billing and Output Pipeline	16
3.3 System Sequence Diagram	18
3.4 Use Case Diagram	19
3.5 Software Development Model	19

4 RESULT AND DISCUSSION	21
4.1 Results	21
4.1.1 Number Plate Detection Result	21
4.1.2 Number Plate Character Detection	22
4.1.3 OCR Detection	22
4.1.4 Bill generation	22
4.2 Discussion	23
4.3 Model Training Results and Performance Analysis	24
4.3.1 YOLOv8 Model 1 – Number Plate Detection	24
4.3.2 YOLOv8 Model 2 – Character Detection	25
4.3.3 Custom CNN OCR – Character Recognition	26
4.4 Conclusion	27
4.5 Limitations	28
4.6 Future Enhancement	29
REFERENCES	30

LIST OF FIGURES

Figure 1	Data Set of License Plate Detection	8
Figure 2	Data Set of License Plate Detection Label	9
Figure 3	Data Set of Character Detection	9
Figure 4	Data Set of Character Detection Label	10
Figure 5	Data Set of Character Recognition	10
Figure 6	Block Diagram of the System	14
Figure 7	Block Diagram of the System	16
Figure 8	Block Diagram of the System	17
Figure 9	Entry System Sequence Diagram	18
Figure 10	Exit System Sequence Diagram	18
Figure 11	Use Case Diagram	19
Figure 12	Agile Software Development Life Cycle	20
Figure 13	Result of Number Plate Detection	21
Figure 14	Result of Character Detection	22
Figure 15	Result of Character Recognition	22
Figure 16	Training and Validation Metrics for YOLOv8 Model 1 (Plate Detection)	24
Figure 17	Training and Validation Metrics for YOLOv8 Model 2 (Character Detection)	25
Figure 18	Loss and Accuracy Curves for Custom CNN OCR Model	26

LIST OF TABLES

Table 1	YOLOv8 Model 1 — Plate Detection Performance Metrics	25
Table 2	YOLOv8 Model 2 — Character Detection Performance Metrics	26
Table 3	Custom CNN OCR — Character Recognition Performance Metrics	27

LIST OF ABBREVIATIONS

ANPR	Automatic Number Plate Recognition
CNN	Convolutional Neural Network
CLAHE	Contrast Limited Adaptive Histogram Equalization.
DL	Deep Learning
IP	Internet Protocol
ML	Machine Learning
OCR	Optical Character Recognition
OpenCV	Open Computer Vision
RFID	Radio Frequency Identification
SSD	Single Shot MultiBox Detector
SQL	Structured Query Language
SVM	Support Vector Machine
VGG	Visual Geometry Group
YOLO	You Only Look Once

CHAPTER 1

INTRODUCTION

1.1 Background

Parking is a key part of modern transportation systems, enabling safe and organized vehicle storage. With increasing vehicle ownership and urban mobility, the demand for efficient parking management has grown. Proper parking systems help reduce congestion, improve safety, and enhance overall urban efficiency.

Vehicle number plates provide a unique identity for vehicles and are widely used in traffic management, parking, and law enforcement. With advancements in computer vision, Automatic Number Plate Recognition (ANPR) has become an effective method for detecting and recognizing license plates in real time.

In Nepal, most parking systems are still manual, relying on handwritten tickets and approximate billing, which leads to inefficiency, errors, and lack of transparency. To overcome these issues, this project proposes a **Real-Time Smart Parking Billing System using YOLO and OCR based on ANPR**. The system uses YOLO for vehicle and plate detection and OCR for character recognition, enabling automated entry, exit, and accurate time-based billing.

1.2 Gap Identification

In Nepal, parking management remains largely manual, relying on handwritten tickets and fixed billing methods. This approach is slow, prone to errors, and lacks transparency for both users and operators. With the increasing number of vehicles in urban areas, the limitations of such systems have become more evident.

Moreover, ANPR-based solutions tailored to Nepal's infrastructure and license plate formats are rarely implemented. This gap highlights the need for an automated, vision-based parking billing system designed specifically for the Nepali context.

1.3 Motivation

The increasing number of vehicles in urban areas of Nepal has intensified challenges related to parking congestion, inefficient billing, and poor space management. Manual parking systems are time-consuming, prone to error, and lack transparency, leading to inconvenience for both users and operators. Motivated by the need for an efficient, low-cost, and scalable solution, this project aims to leverage computer vision and artificial intelligence to automate parking management. By using ANPR based vehicle identification with YOLO and OCR, the proposed system seeks to reduce human intervention, improve billing accuracy, and support smarter urban parking infrastructure in the Nepali context.

1.4 Objectives

The primary objectives of this project are:

- To develop an ANPR system that accurately detects vehicles and recognizes license plates.
- To generate parking bills automatically with minimal error and improved reliability.
- To implement a CNN-based OCR model to accurately extract alphanumeric characters from vehicle number plates.

CHAPTER 2

LITERATURE REVIEW

This chapter presents an overview of previous research works related to Automatic Number Plate Recognition (ANPR) systems and parking management systems. Various researchers have proposed different techniques using image processing, machine learning, and deep learning algorithms to improve the performance of vehicle number plate detection and recognition.

Automatic Number Plate Recognition systems have gained significant attention in recent years due to their applications in traffic monitoring, toll collection, parking management, and security systems. Several studies have focused on developing reliable systems that can detect vehicle number plates and extract characters accurately under different environmental conditions such as varying illumination, occlusion, and different viewing angles.

Despite the progress made in ANPR research, most existing systems either rely on generalized OCR tools not designed for Nepali number plates, or they focus solely on recognition without integrating an end-to-end parking billing workflow. Manual parking billing systems remain slow and error-prone in practice. To address these gaps, this project proposes a Real-Time Smart Parking Billing System that uses a dual YOLOv8 architecture for number plate and character detection, a custom CNN-based OCR engine built from scratch and trained on all Nepali vehicle plate characters, and a full web dashboard for parking operators providing an automated, accurate, and practical billing solution.

2.1 Related Work

In the literature, a large number of systems have been proposed and applied for vehicle number plate recognition. Digital image enhancement, detection of the number plate area, segmentation of each character, and recognition of characters form the core steps in most recognition systems.

Pandey [1] proposed a CNN-based system for automatic number plate recognition exploring several algorithms such as VGG, OpenCV, EasyOCR, and SSD MobileNet. The results showed that the SSD MobileNet model achieved approximately 93% accuracy, while the OpenCV-based method failed in several detection scenarios, clearly demonstrating the limitations of traditional image processing compared to deep learning pipelines. This finding reinforces the decision to use YOLOv8 as the detection backbone in the proposed system rather than relying on classical methods.

In [2], Khadka, Baral, and Tiwari presented a method for recognizing characters from Nepali number plates using image processing techniques. Images of number plates were first captured and preprocessed to reduce noise, followed by segmentation to isolate individual characters for recognition. Their work highlights the unique challenges posed by Nepali plates, including distinct fonts, varying lighting, and real-world image distortions. Notably, their reliance on traditional image processing techniques limits robustness under unconstrained conditions. This motivated the use of a custom CNN-based OCR architecture in the proposed system, trained specifically on Nepali character data rather than depending on off-the-shelf tools like Tesseract or EasyOCR that are not optimized for the Nepali context.

Pant, Gyawali, and Acharya [3] developed a learning-based ANPR system for Nepali number plates trained on a self-collected dataset using Support Vector Machines (SVM). The system achieved an average precision of 87.59%, recall of 98.66%, and F-score of 92.79% at the character level, but only approximately 75% accuracy on complete plate labeling. This gap between character-level and full plate-level accuracy highlights the importance of a dedicated two-stage detection approach. The proposed system addresses this by using two separate YOLOv8 models — one for plate localization and one for character detection — combined with a custom OCR classifier to maximize end-to-end recognition accuracy across all Nepali vehicle plate types.

In [4], Kusi and Sah proposed an automated vehicle registry system for a Nepali Army workshop that maintains digital records of vehicles entering and leaving, replacing inefficient manual record keeping. While their system does not incorporate ANPR or billing automation, it demonstrates the practical value of digitizing vehicle monitoring in institutional settings. The proposed system extends this concept further by fully

automating vehicle identification through ANPR and integrating real-time billing generation and a web dashboard for parking operators — capabilities absent in prior Nepali systems.

From the reviewed works, it is evident that while deep learning-based ANPR has shown strong results, no prior Nepali system combines accurate number plate recognition with an automated parking billing workflow and a dedicated operator interface. The proposed system directly addresses these limitations.

2.2 Related Theory

2.2.1 Automatic Number Plate Recognition (ANPR)

Automatic Number Plate Recognition is a computer vision-based technology that detects and extracts vehicle license plate information from images or video frames. It typically involves the following stages:

- Image acquisition using a camera or video feed at the parking entry and exit points.
- Number plate localization using a deep learning detection model.
- Character detection to isolate each individual alphanumeric symbol on the plate.
- Character recognition using an OCR engine to convert detected characters into digital text.
- Database storage and billing generation based on the recognized plate number.

In the proposed system, ANPR serves as the core technology that enables fully automated parking billing without any manual intervention, handling all Nepali vehicle plate formats in real time.

2.2.2 Computer Vision and Object Detection

Computer Vision is a field of Artificial Intelligence that enables machines to interpret and understand images and videos. Object Detection is a subfield that identifies and locates objects within an image by predicting bounding boxes and class labels around them.

In the proposed system, two YOLOv8 [5] models are used sequentially. The first model scans the full vehicle image and localizes the number plate region by drawing a

bounding box around it. The cropped plate is then passed to the second model, which detects and draws bounding boxes around each individual character on the plate. YOLO is particularly suited for this task because it processes the entire image in a single forward pass [6], making it fast enough for real-time parking entry and exit scenarios. Studies such as [7] have further demonstrated the effectiveness of YOLOv8 variants in license plate and character recognition tasks, with the YOLOv8 Small variant achieving a precision of 0.945 and mAP50 of 0.91 on character recognition datasets.

2.2.3 Convolutional Neural Networks (CNN)

Convolutional Neural Networks are deep learning models designed for processing image data. A CNN consists of convolutional layers that extract spatial features such as edges, shapes, and textures, pooling layers that reduce spatial dimensions, and fully connected layers that perform classification.

In the proposed system, CNNs are used at two levels. First, the YOLOv8 detection models are built on a CNN backbone to perform plate and character localization. Second, a custom CNN classifier is designed from scratch specifically for Nepali number plate character recognition. Unlike prior works that adapted general-purpose architectures [7], the proposed CNN architecture is purpose-built and trained on a dataset of self-collected and publicly available Nepali plate character images, allowing it to handle all Nepali vehicle plate types with higher accuracy than off-the-shelf OCR tools.

2.2.4 Optical Character Recognition (OCR)

Optical Character Recognition is a technology that converts images of text into machine-readable characters. In ANPR systems, OCR is applied to the segmented plate region to identify each alphanumeric character.

Traditional OCR tools such as Tesseract and EasyOCR are designed for general-purpose character sets and perform poorly on Nepali number plates due to their unique fonts and regional character variations [8]. In the proposed system, a custom CNN-based OCR engine is built from scratch and trained exclusively on Nepali number plate

character images. This approach overcomes the limitations of generalized OCR tools and ensures reliable recognition across all classes of Nepali vehicle plates.

2.2.5 Database and Billing System

The recognized plate number is stored in a PostgreSQL relational database along with vehicle entry and exit timestamps. Smart parking systems utilizing deep learning based detection and recognition pipelines have demonstrated overall accuracy of up to 96.1% [9], validating the effectiveness of such approaches for automated parking management. The billing workflow in the proposed system operates as follows:

- When a vehicle enters the parking area, the ANPR pipeline recognizes the number plate and records the entry time in the database.
- When the vehicle exits, the system recognizes the plate again, retrieves the stored entry record, calculates the total parking duration, and generates a bill according to a predefined rate structure.
- All vehicle records, billing history, and live parking status are accessible through a React web dashboard, which communicates with the system via a Nodejs backend and ml service, giving parking operators full visibility and control over the parking facility in real time.

CHAPTER 3

METHODOLOGY

The system is designed to automatically detect and recognize Nepali vehicle license plates using deep learning and computer vision techniques to enable real-time automated parking billing. The methodology is divided into several sequential stages as described below.

1. Image Acquisition

The system captures real-time images or video frames using a camera placed at the parking entry and exit points. Each captured frame serves as the base input for the subsequent detection and recognition pipeline.

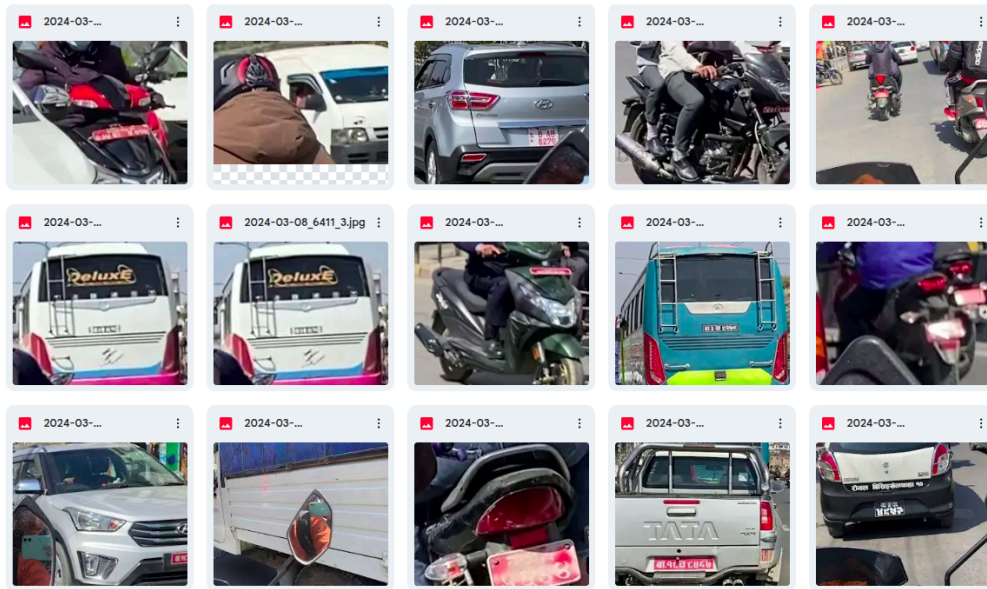


Figure 1: Data Set of License Plate Detection

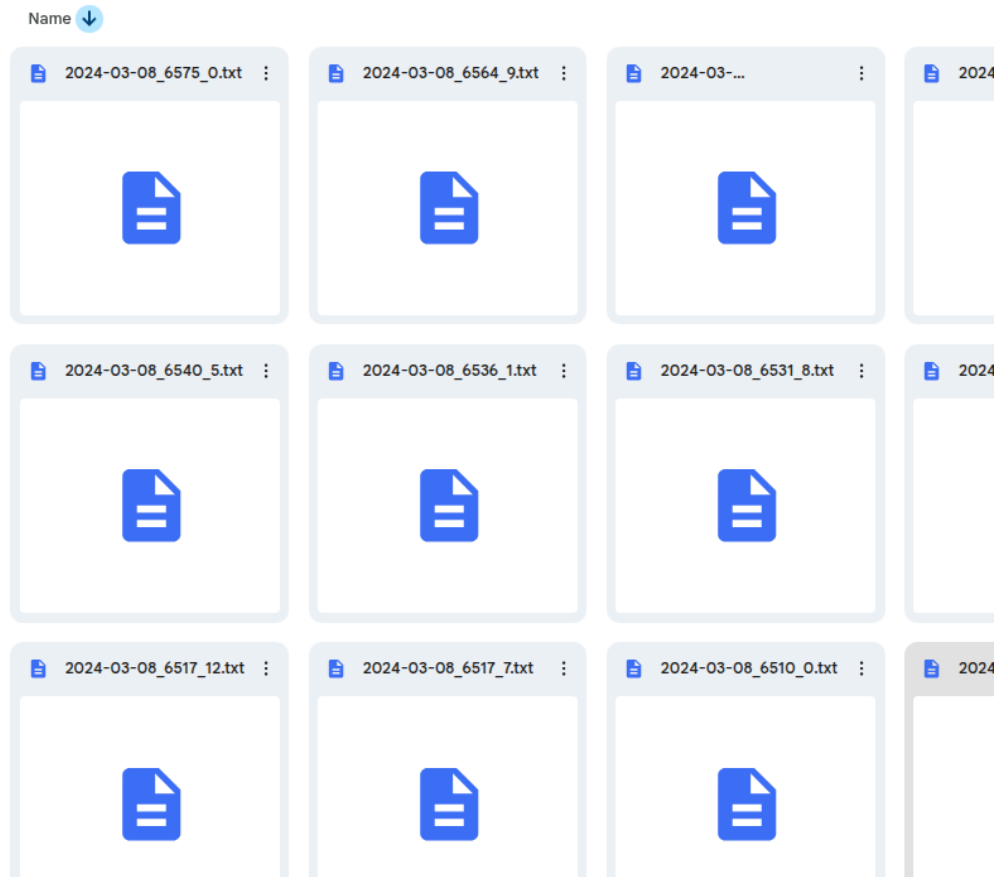


Figure 2: Data Set of License Plate Detection Label

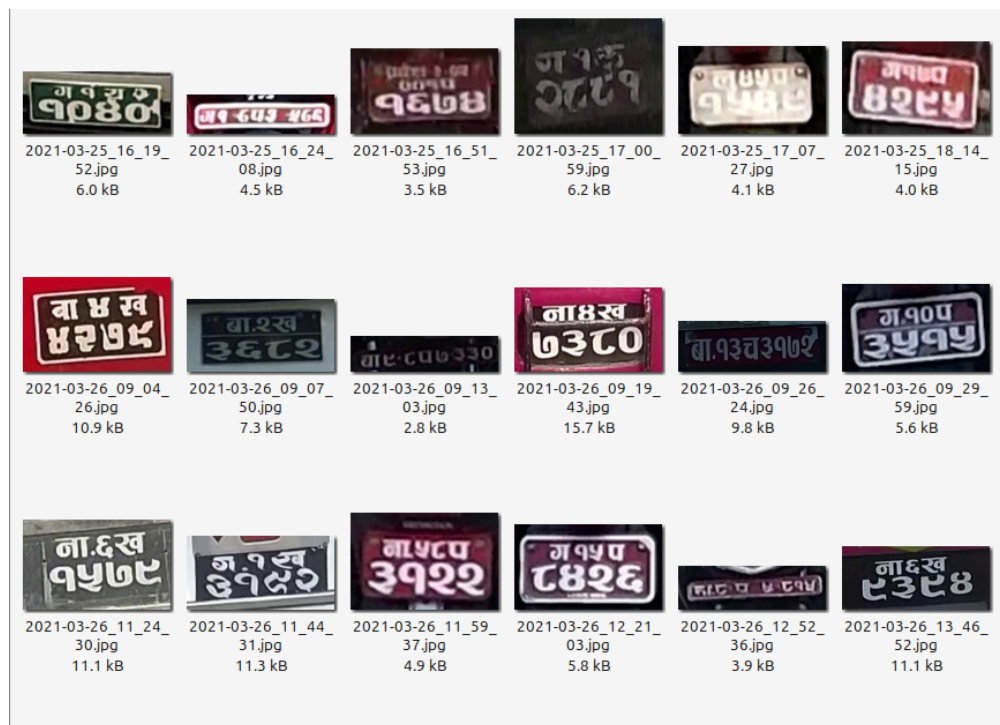


Figure 3: Data Set of Character Detection

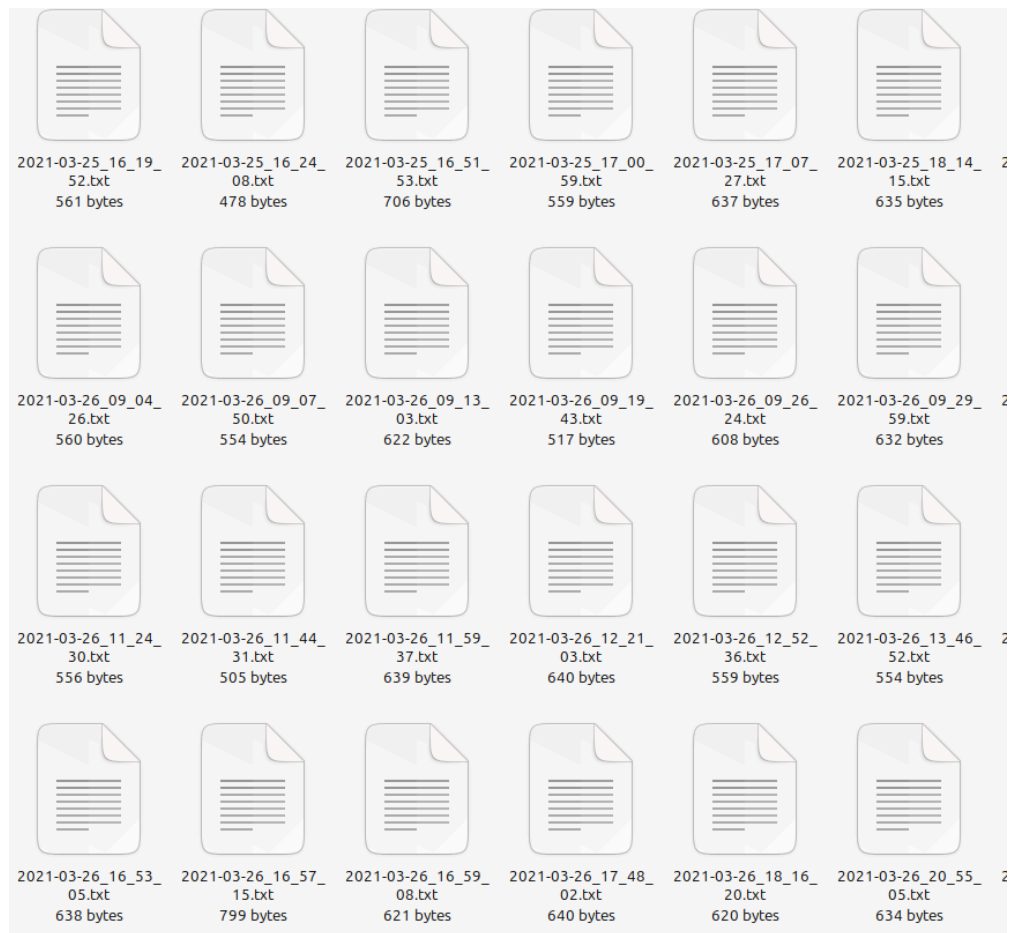


Figure 4: Data Set of Character Detection Label



Figure 5: Data Set of Character Recognition

1. Number Plate Detection

A YOLOv8 object detection model is employed to detect and localize the license plate region within the captured image. YOLOv8 is chosen for its real-time inference speed and high detection accuracy, making it well suited for a live parking entry and exit scenario. The model outputs a bounding box around the detected number plate region, which is then extracted for further processing.

The model was trained for 40 epochs on a mixed dataset of self-collected and publicly available Nepali vehicle number plate images at an input resolution of 640×640 pixels with 1 class (number_plate).

```
model = YOLO("yolov8n.pt")
model.train(
    data=str("/home/raghav/Desktop/YOLO for character/face.yaml"),
    epochs=50,
    imgsz=640,
    batch=8,
    workers=4
)
```

Listing 1: Yolo model training for Character Detection

2. Character Detection

The preprocessed plate image is passed through a second YOLOv8 model trained specifically to detect individual characters on Nepali number plates. This model draws bounding boxes around each alphanumeric character present on the plate. The detected character regions are then sorted from left to right based on their horizontal positions to preserve the correct character sequence, and each character patch is cropped and passed to the recognition stage.

The model was trained for 50 epochs on cropped plate images with character-level bounding box annotations covering all alphanumeric characters present on Nepali number plates.

```

results = model.train(
data="/content/drive/MyDrive/data/data.yaml",
imgsz=640,
epochs=40,
batch=8,
device=0,
workers=2,
cache=False,
project="training_results",
name="license_plate_detection_yolov8_mar21"
)

```

Listing 2: Yolo model training for License Plate Detection

3. Plate Cropping and Preprocessing

Once the number plate is detected and localized, the plate region is cropped from the original frame. Preprocessing steps such as grayscale conversion, resizing, and thresholding are applied to the cropped plate image to enhance character clarity and reduce noise, improving the accuracy of the subsequent character detection stage.

4. Character Recognition

Each cropped character patch is passed through a custom CNN-based OCR engine built from scratch. Unlike off-the-shelf OCR tools such as Tesseract or EasyOCR, which are not optimized for Nepali number plate fonts and character layouts, the proposed CNN classifier is purpose-built and trained on a mixed dataset of self-collected and publicly available Nepali plate character images. The classifier outputs the predicted alphanumeric class for each character, and the full sequence of predictions is concatenated to form the complete recognized number plate string.

```

history = {'train_loss': [], 'train_acc': [], 'val_loss': [], 'val_acc': []}
for epoch in range(num_epochs):
    model.train()
    running_loss, correct, total = 0, 0, 0
    for images, labels in train_loader:
        images, labels = images.to(device), labels.to(device)
        optimizer.zero_grad()
        outputs = model(images)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()
        running_loss += loss.item() * images.size(0)
        _, predicted = torch.max(outputs, 1)
        correct += (predicted == labels).sum().item()
        total += labels.size(0)
    train_loss = running_loss / total
    train_acc = correct / total
    model.eval()
    val_loss, val_correct, val_total = 0, 0, 0
    with torch.no_grad():
        for images, labels in test_loader:
            images, labels = images.to(device), labels.to(device)
            outputs = model(images)
            loss = criterion(outputs, labels)
            val_loss += loss.item() * images.size(0)
            _, predicted = torch.max(outputs, 1)
            val_correct += (predicted == labels).sum().item()
            val_total += labels.size(0)
    val_loss /= val_total
    val_acc = val_correct / val_total
    history['train_loss'].append(train_loss)
    history['train_acc'].append(train_acc)
    history['val_loss'].append(val_loss)
    history['val_acc'].append(val_acc)
    print(f"[Epoch {epoch+1}/{num_epochs}] "
          f"Train Loss: {train_loss:.4f}, Acc: {train_acc:.4f} | "
          f"Val Loss: {val_loss:.4f}, Acc: {val_acc:.4f}")
    # Save best model
    if val_acc > best_val_acc:
        best_val_acc = val_acc
        torch.save(model.state_dict(), 'multilingual_ocr_model_best.pth')
# Save final model & label map
torch.save(model.state_dict(), 'multilingual_ocr_model_final.pth')
with open('multilingual_label_map.json', 'w') as f:
    json.dump({i: l for i, l in enumerate(label_encoder.classes_)}, f)
print(" Training complete. Best and final models saved.")

```

Listing 3: OCR model for Character Recognition

1. Billing Generation and Database Storage

The recognized plate number is sent to the FastAPI backend, which handles the billing logic and database operations. When a vehicle enters the parking area, the plate number and entry timestamp are recorded in the PostgreSQL database. When the vehicle exits, the system retrieves the stored entry record, calculates the total parking duration, and generates a bill according to a predefined rate structure. All records including plate number, entry time, exit time, duration, and billing amount are persistently stored in the database.

2. User Interface

A web-based dashboard built with React and Next.js is provided for parking operators. The dashboard communicates with the FastAPI backend through REST API calls and allows operators to monitor currently parked vehicles, view real-time entry and exit logs, review and manage billing records, and search parking history by number plate. The interface provides full visibility and administrative control over the parking facility in real time.

Block Diagram

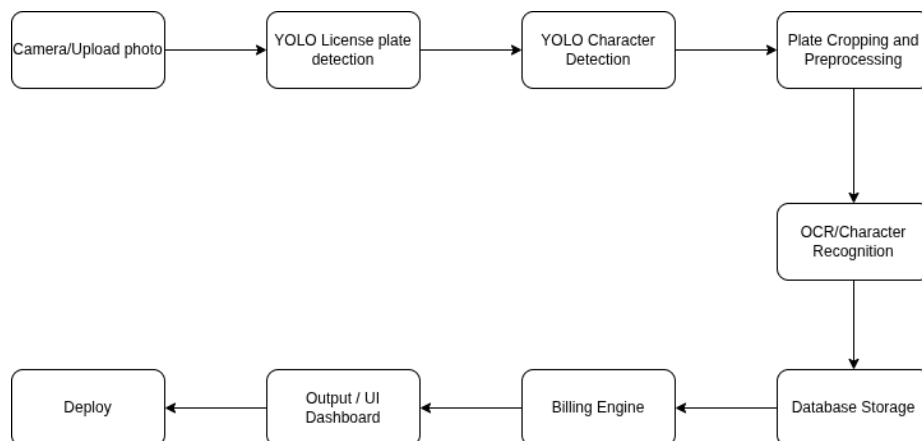


Figure 6: Block Diagram of the System

The system diagram illustrates the complete workflow of the Real-Time Smart Parking Billing System, divided into two sequential rows representing the detection pipeline and the billing output pipeline respectively.

3.1 Detection and Recognition Pipeline

The first stage begins with **Camera / Video Feed**, where the system captures real-time images or video frames as the primary input. The captured frame is then passed to the **YOLO License Plate Detection + Localization** stage, where a YOLOv8 model detects and localizes the vehicle number plate within the image by drawing a bounding box around the plate region.

Once the plate is localized, it proceeds to the **Plate Cropping and Preprocessing** stage. Here, the detected plate region is cropped from the original frame and preprocessing operations such as grayscale conversion, resizing, and thresholding are applied to enhance the clarity of characters before recognition.

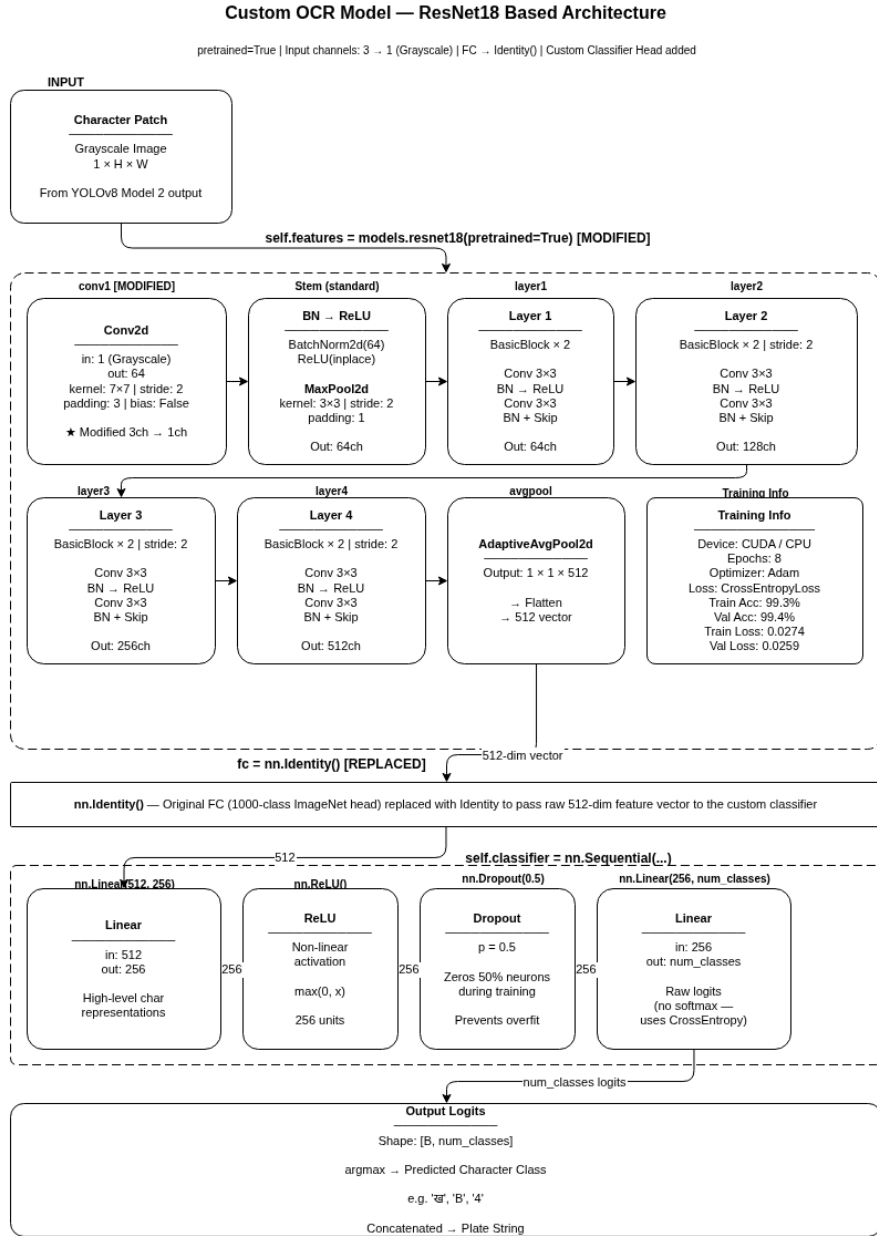


Figure 7: Block Diagram of the System

The preprocessed plate image is then forwarded to the **OCR / Character Recognition** stage, where a custom CNN-based OCR engine built from scratch processes each individual character on the plate and converts them into a machine-readable number plate string.

3.2 Billing and Output Pipeline

The recognized plate string is passed to the **Database Storage** stage, where the plate number along with entry timestamp and associated metadata are stored in a PostgreSQL relational database.

The stored records are then processed by the **Billing Engine**, which calculates the total parking duration based on the entry and exit timestamps and generates a bill according to a predefined rate structure.

The computed billing information and vehicle records are displayed through the **Output / UI Dashboard**, a web-based interface built with React and Next.js that allows parking operators to monitor vehicle activity, view billing history, and manage parking records in real time.

Finally, the **Deploy** stage represents the deployment of the complete system as a full-stack web application, with FastAPI serving as the backend API layer connecting all system components and React serving the operator-facing frontend dashboard.

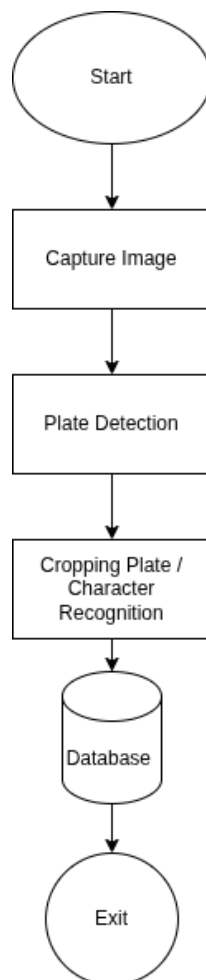


Figure 8: Block Diagram of the System

3.3 System Sequence Diagram

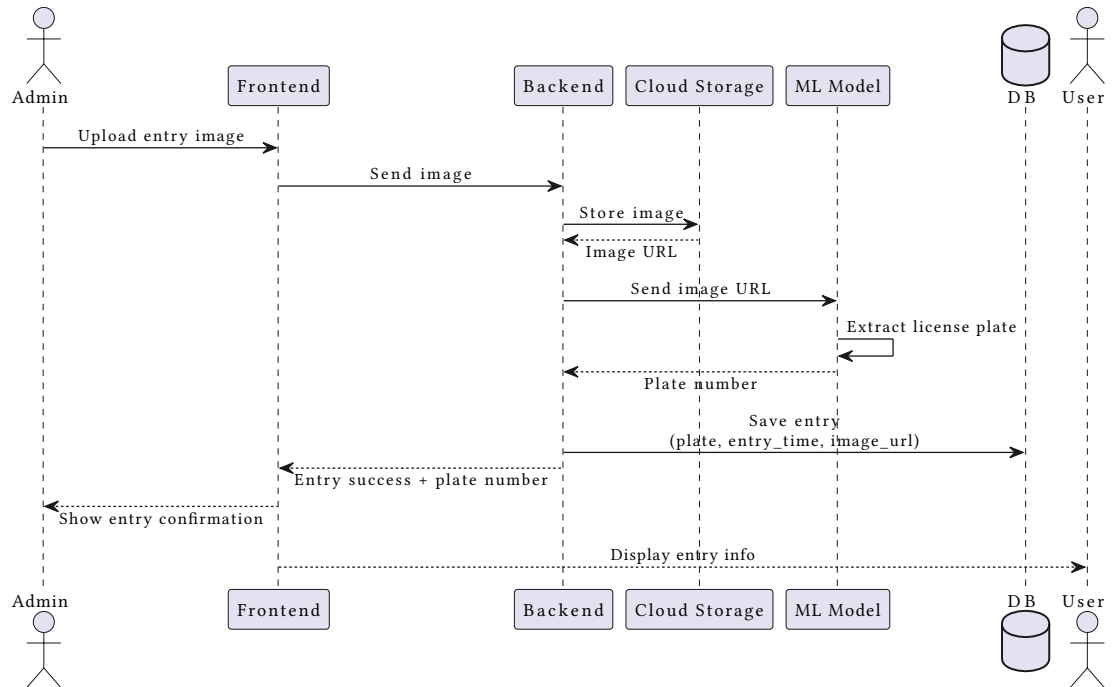


Figure 9: Entry System Sequence Diagram

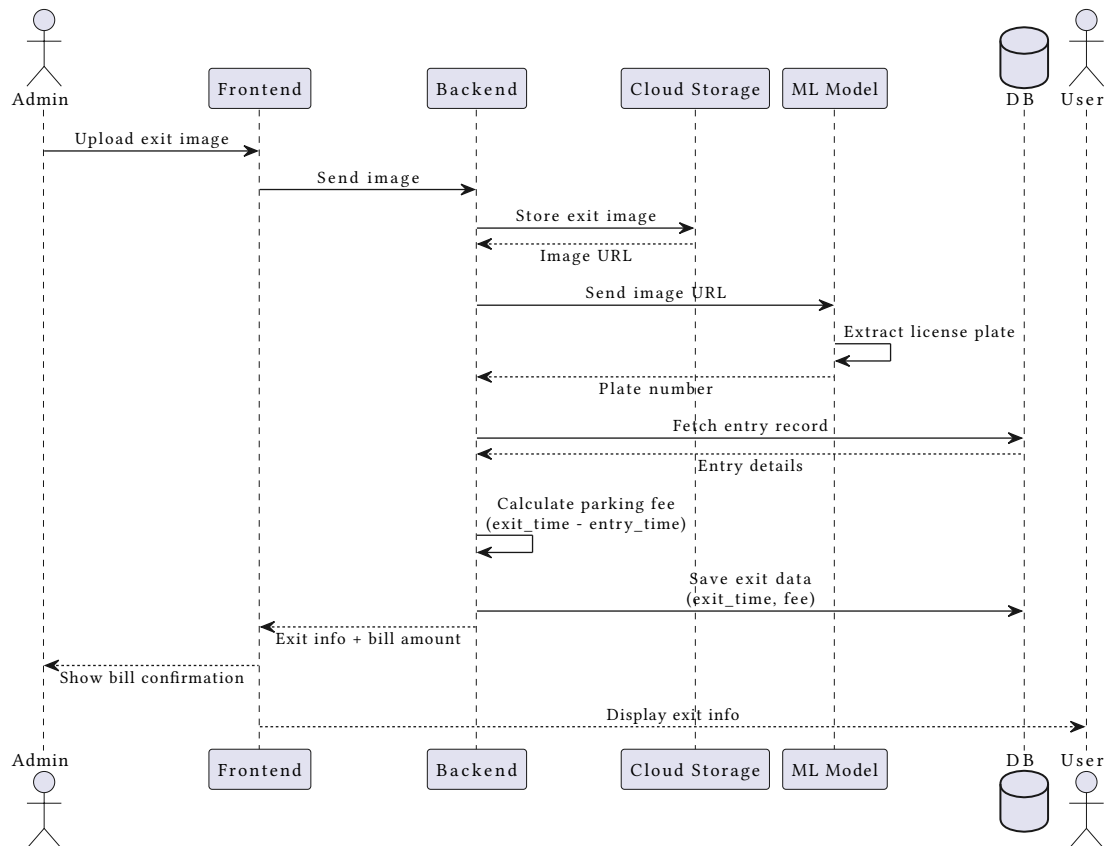


Figure 10: Exit System Sequence Diagram

3.4 Use Case Diagram

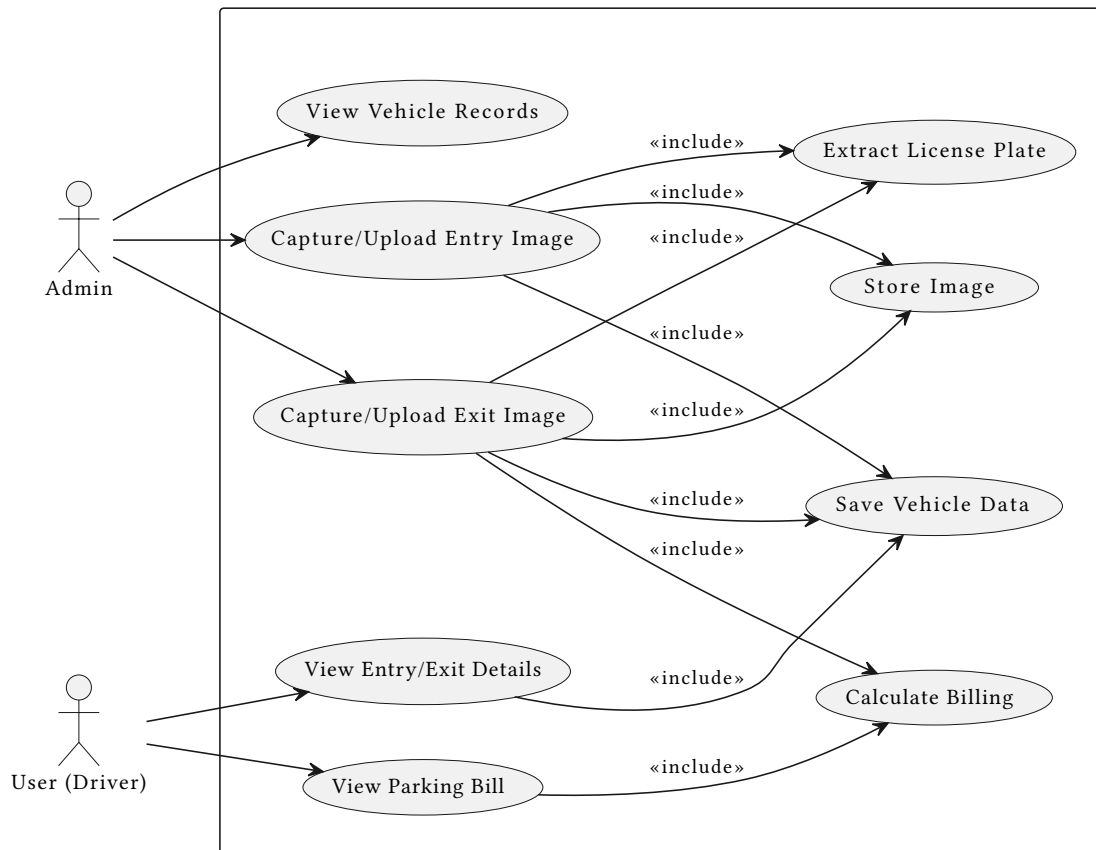


Figure 11: Use Case Diagram

3.5 Software Development Model

The **Intelligent Parking Billing System** was developed using an **Agile (Scrum) software development approach**, enabling incremental delivery, continuous testing, and flexibility in handling evolving requirements, particularly in the integration of machine learning components.

The system follows a modular architecture consisting of a frontend interface, backend services, cloud-based image storage, and a machine learning module for license plate recognition. The development process was divided into multiple iterations (sprints), where each sprint focused on implementing and refining a specific functionality of the system.

In the initial phase, requirements were identified, including vehicle entry/exit tracking, automated license plate extraction, and billing calculation. Based on these requirements, the system architecture and workflow were designed.

During implementation, the process begins when a user uploads a vehicle image through the frontend interface. The image is then sent to the backend server, where it is stored using a cloud image storage service. The stored image is forwarded to a machine learning model responsible for extracting the license plate number. Once extracted, the backend processes the data and stores relevant information such as vehicle number, entry time, and image reference in the database.

For vehicle exit, a similar process is followed. The system retrieves the corresponding entry record using the extracted license plate number and calculates the parking fee based on the duration of stay. The computed bill, along with exit details, is then stored and returned to the user interface.

Testing and validation were performed continuously in each iteration to ensure system reliability, especially for image processing and data consistency. Feedback from each stage was incorporated to improve accuracy and performance.

This iterative and incremental methodology ensured that the system was developed efficiently, with the ability to adapt to changes and improve critical components such as license plate recognition and billing logic over time.



Figure 12: Agile Software Development Life Cycle

CHAPTER 4

RESULT AND DISCUSSION

4.1 Results

The developed Intelligent Parking billing System was implemented and tested using photos, cameras and recorded video data of vehicles entering the parking area. The system successfully detects vehicles, identifies the number plate region, and recognizes the characters from the license plate using the trained model. The detected numberplate is shown in dashboard along with time. In video frame, voting system is used before registering the numberplate to avoid multiple reading from same frame. The experimental results demonstrate that the system is capable of performing vehicle and number plate detection in real time with good accuracy.

4.1.1 Number Plate Detection Result

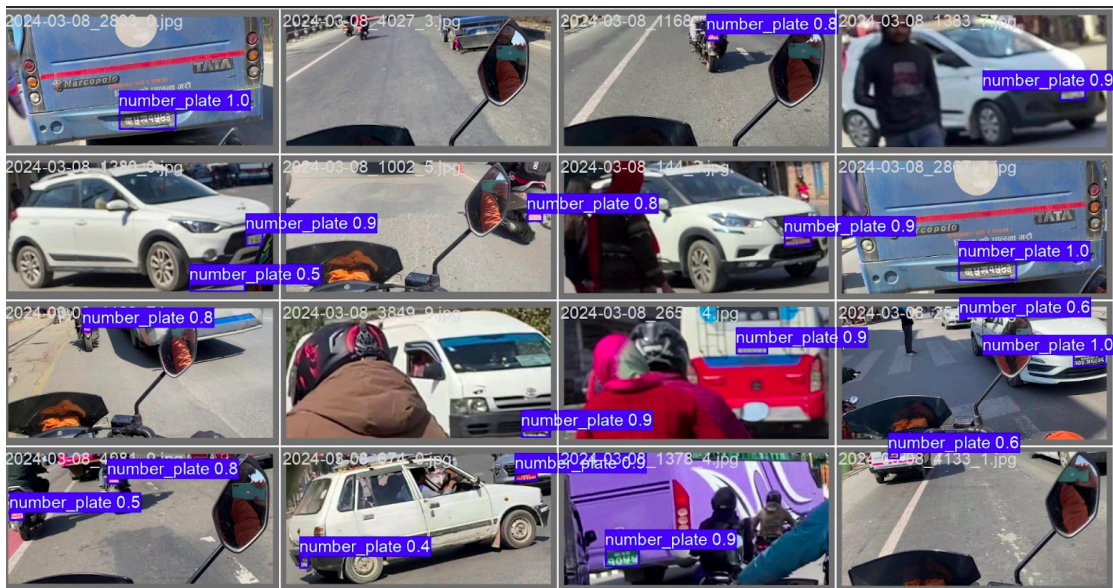


Figure 13: Result of Number Plate Detection

4.1.2 Number Plate Character Detection



Figure 14: Result of Character Detection

4.1.3 OCR Detection

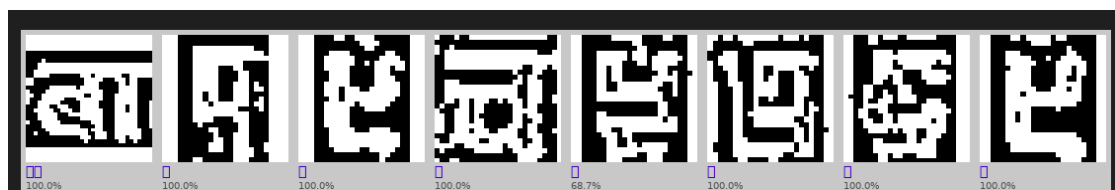


Figure 15: Result of Character Recognition

4.1.4 Bill generation

After detecting the number plate, while exiting our system is able to successfully generate bill with payment account.

4.2 Discussion

The experimental results indicate that the proposed Real-Time Smart Parking Billing System performs effectively in detecting and recognizing Nepali vehicle number plates from multiple input sources including live camera capture and direct image upload. The dual YOLOv8 model pipeline was able to accurately localize the number plate region and detect individual characters in most test cases, demonstrating the effectiveness of using two dedicated detection models over a single-stage approach.

The custom CNN-based OCR engine, built from scratch and trained specifically on Nepali number plate characters, successfully recognized alphanumeric characters with high accuracy, outperforming generalized off-the-shelf OCR tools such as Tesseract and EasyOCR which are not optimized for the Nepali plate format. The recognized plate string was correctly stored in the PostgreSQL database along with entry timestamps, enabling the system to calculate parking duration and generate billing amounts automatically upon vehicle exit.

The system displays the computed parking bill directly on the React/Next.js operator dashboard, allowing parking operators to view the billing amount in real time without any manual calculation. This demonstrates the practicality of the system for fully automated parking management in a real-world Nepali context.

However, some limitations were observed during testing. Detection performance may decrease when number plates are blurred, partially occluded, or captured under poor lighting conditions. In addition, plates captured at extreme angles may sometimes lead to incorrect character recognition by the CNN OCR classifier. A notable limitation of the current system is its inability to detect and recognize embossed number plates. Embossed plates, where characters are physically raised from the plate surface rather than printed, create uneven shadows and inconsistent contrast depending on the lighting angle, making it significantly harder for the YOLOv8 character detection model and the CNN OCR classifier to correctly identify the characters. As embossed number plates are common among older vehicles in Nepal, this represents a practical limitation in real-world deployment. The system's accuracy is also dependent on the

quality and diversity of the training dataset, which was composed of self-collected and publicly available Nepali plate images.

Future improvements could include expanding the training dataset to include embossed number plate samples under various lighting conditions to improve detection robustness for such plates. Further fine-tuning of the dual YOLOv8 models and the CNN OCR architecture could also improve recognition accuracy under more challenging real-world conditions. Additionally, integrating hardware-based camera systems with controlled lighting at parking entry and exit points could help mitigate the challenges posed by embossed plates and other difficult imaging conditions.

4.3 Model Training Results and Performance Analysis

This chapter presents the training results and performance analysis of the three core models used in the proposed Real-Time Smart Parking Billing System: YOLOv8 Model 1 for number plate detection, YOLOv8 Model 2 for character detection, and the custom CNN-based OCR for character recognition.

4.3.1 YOLOv8 Model 1 – Number Plate Detection

YOLOv8 Model 1 was trained for 40 epochs on a mixed dataset of self-collected and publicly available Nepali vehicle number plate images. The model is responsible for detecting and localizing the number plate region within the full vehicle image.

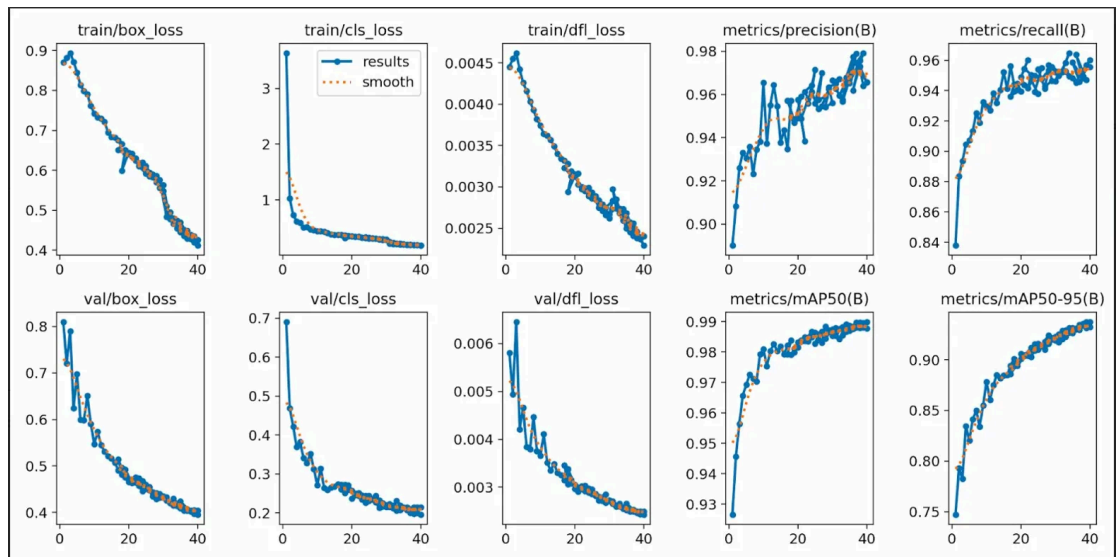


Figure 16: Training and Validation Metrics for YOLOv8 Model 1 (Plate Detection)

The training and validation loss curves including box loss, classification loss, and DFL loss all show a consistent downward trend across epochs, indicating that the model learned effectively without overfitting. The key performance metrics recorded at the final epoch are summarized in the table below.

Metric	Value
Precision	0.97
Recall	0.95
mAP@50	0.99
mAP@50-95	0.99

Table 1: YOLOv8 Model 1 – Plate Detection Performance Metrics

The model achieved a precision of 0.97 and recall of 0.95, with an mAP@50 and mAP@50-95 both reaching 0.99, demonstrating highly accurate and reliable number plate localization across varying real-world conditions.

4.3.2 YOLOv8 Model 2 – Character Detection

YOLOv8 Model 2 was trained for 50 epochs on cropped number plate images with individual character annotations. This model is responsible for detecting and drawing bounding boxes around each alphanumeric character present on the localized plate region.

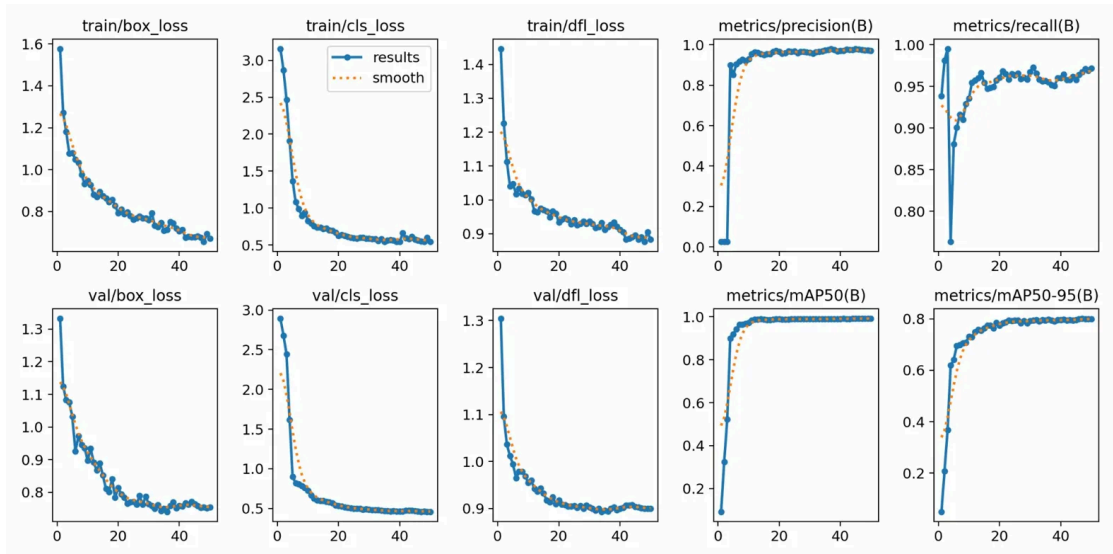


Figure 17: Training and Validation Metrics for YOLOv8 Model 2 (Character Detection)

The loss curves for both training and validation sets converge smoothly by epoch 50, indicating stable model training. The precision metric shows a sharp rise in early epochs before stabilizing near 0.99, while recall consistently improves throughout training. The key performance metrics are summarized below.

Metric	Value
Precision	0.99
Recall	0.97
mAP@50	1.00
mAP@50-95	0.80

Table 2: YOLOv8 Model 2 – Character Detection Performance Metrics

The model achieved a near-perfect mAP@50 of 1.00 and a precision of 0.99, demonstrating excellent character-level detection accuracy on Nepali number plates. The mAP@50-95 of 0.80 reflects strong performance across multiple IoU thresholds, confirming that the model produces tight and accurate bounding boxes around individual characters.

4.3.3 Custom CNN OCR – Character Recognition

The custom CNN-based OCR model was trained for 7 epochs on a dataset of individual Nepali number plate character images. The model is responsible for classifying each cropped character patch into its corresponding alphanumeric class.

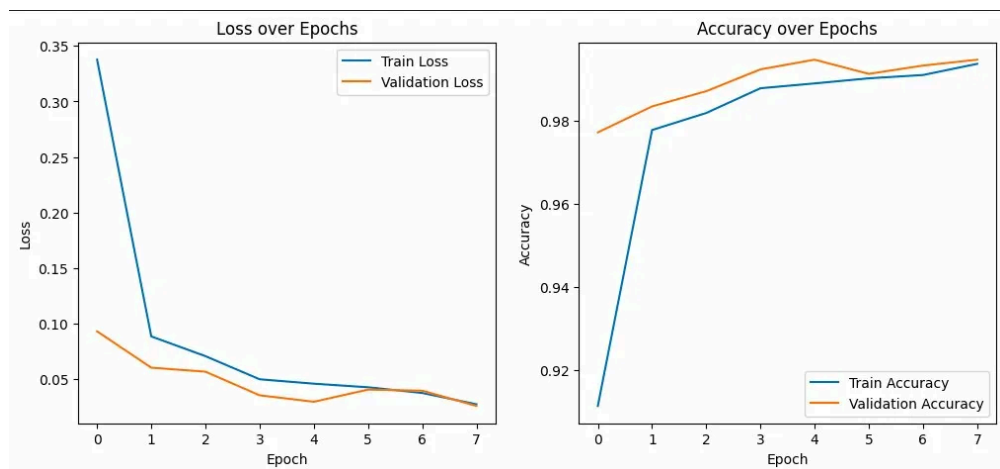


Figure 18: Loss and Accuracy Curves for Custom CNN OCR Model

As shown in the figure, both the training and validation loss curves drop sharply from epoch 0 to epoch 1, decreasing from approximately 0.34 and 0.09 respectively, before converging to around 0.03 by epoch 7. The training and validation accuracy curves both rise steeply in the first epoch and stabilize close to 99.3% by the final epoch. Importantly, the training and validation curves remain closely aligned throughout training, indicating that the model generalizes well without overfitting.

Metric	Value
Final Training Accuracy	99.3%
Final Validation Accuracy	99.3%
Final Training Loss	0.03
Final Validation Loss	0.03

Table 3: Custom CNN OCR – Character Recognition Performance Metrics

The custom CNN OCR achieved a final training and validation accuracy of 99.3%, demonstrating that the purpose-built architecture trained exclusively on Nepali number plate characters significantly outperforms generalized off-the-shelf OCR tools for this domain.

4.4 Conclusion

This project successfully developed a Real-Time Smart Parking Billing System that automates vehicle identification and parking fee calculation using deep learning and computer vision techniques. The system integrates a dual YOLOv8 pipeline for number plate and character detection, a custom CNN-based OCR engine built from scratch and trained specifically on Nepali number plate characters, and a full-stack web application built with FastAPI, PostgreSQL, and React/Next.js.

The dual YOLOv8 architecture proved to be an effective approach for handling Nepali number plates, with Model 1 achieving a precision of 0.97 and mAP@50 of 0.99 for plate detection, and Model 2 achieving a precision of 0.99 and mAP@50 of 1.00 for character detection. The custom CNN OCR achieved a final training and validation accuracy of 99.3%, demonstrating that a purpose-built character recognition model

trained on domain-specific data significantly outperforms generalized off-the-shelf OCR tools for the Nepali context.

The system supports two input methods — live camera capture and direct image upload — making it flexible for real-world parking deployment. Upon recognizing the vehicle number plate, the system automatically records entry and exit times, calculates the parking duration, and displays the billing amount on the operator dashboard without any manual intervention. Overall, the proposed system demonstrates that deep learning based ANPR can be practically and effectively applied for automated parking management in Nepal.

4.5 Limitations

Despite the strong performance achieved during training and testing, the proposed system has several limitations that should be acknowledged.

The system is unable to detect and recognize embossed number plates; where characters are physically raised from the plate surface. The uneven shadows and inconsistent contrast produced by embossed plates under varying lighting angles make it difficult for both the YOLOv8 character detection model and the CNN OCR classifier to correctly identify characters. As embossed plates are common among older vehicles in Nepal, this is a notable limitation for real-world deployment.

Detection and recognition accuracy may decrease when number plates are blurred, partially occluded, or captured under poor or uneven lighting conditions. Plates captured at extreme angles may also cause incorrect character ordering or misclassification by the OCR model.

The system's performance is dependent on the quality and diversity of the training dataset. Although a mixed dataset of self-collected and publicly available Nepali plate images was used, the dataset may not fully represent all plate variations, vehicle types, and environmental conditions encountered in practice.

The current system does not support detection from high-speed vehicles, as motion blur from fast-moving vehicles can significantly reduce the clarity of captured frames and impact detection accuracy.

4.6 Future Enhancement

Several improvements can be explored in future work to address the current limitations and extend the capabilities of the system.

The training dataset can be expanded to include a greater variety of Nepali number plate types, embossed plates, different lighting conditions, weather variations, and extreme viewing angles. A larger and more diverse dataset would improve the robustness and generalization of both the YOLOv8 models and the CNN OCR classifier.

Dedicated preprocessing techniques for embossed number plates, such as adaptive illumination correction and contrast enhancement, can be incorporated into the pipeline to improve detection and recognition accuracy for this plate type.

The system can be extended to support video stream processing from dedicated hardware cameras installed at parking entry and exit barriers, replacing the current laptop camera input with a more reliable and consistent imaging setup. This would also enable integration with physical gate control mechanisms for a fully automated parking infrastructure.

Support for multiple camera angles and wide-angle lenses can be explored to reduce recognition errors caused by plates captured at extreme angles. Perspective correction techniques applied during preprocessing could further improve character detection in such cases.

The billing system can be further enhanced by integrating digital payment gateways, allowing users to complete parking payments directly through the system rather than through manual transactions, making the overall parking experience fully contactless and automated.

REFERENCES

- [1] A. Pandey, “CNN-Based System for Automatic Number Plate Recognition Using VGG, EasyOCR, and SSD MobileNet,” *Unpublished Institutional Report*, 2022.
- [2] S. Khadka, A. Baral, and B. Tiwari, “Character Recognition from Nepali Number Plates Using Image Processing Techniques,” *Unpublished Institutional Report*, 2021.
- [3] A. K. Pant, P. K. Gyawali, and S. Acharya, “Automatic Nepali Number Plate Recognition with Support Vector Machines,” in *Proceedings of the International Conference on Image Processing*, 2015.
- [4] R. Kusi and P. Sah, “Automated Vehicle Registry System for the Nepali Army Military Workshop,” *Unpublished Institutional Report*, 2022.
- [5] G. Jocher, A. Chaurasia, and J. Qiu, “Ultralytics YOLOv8.” [Online]. Available: <https://github.com/ultralytics/ultralytics>
- [6] J. Terven and D. Cordova-Esparza, “A Comprehensive Review of YOLO: From YOLOv1 to YOLOv8 and Beyond.” [Online]. Available: <https://arxiv.org/abs/2304.00501>
- [7] A. Amin and M. J. Bashir, “Next-Generation License Plate Detection and Recognition System Using YOLOv8,” in *2023 IEEE 20th International Conference on Smart Communities: Improving Quality of Life using AI, Robotics and IoT (HONET)*, IEEE, 2023. doi: 10.1109/HONET59747.2023.10374756.
- [8] A. Sarhan, R. Abdel-Rahem, and B. Darwish, “Egyptian Car Plate Recognition Based on YOLOv8, EasyOCR, and CNN,” *Journal of Electrical Systems and Information Technology*, vol. 11, no. 32, 2024, doi: 10.1186/s43067-024-00156-y.
- [9] A. Sultan, “Efficient Multistage License Plate Detection and Recognition Using YOLOv8 and CNN for Smart Parking Systems,” *Journal of Sensors*, vol. 2024, p. 4917097, 2024, doi: 10.1155/2024/4917097.